

STACKPROBE

10-Week Execution Plan

Ship a production-grade async GitHub analyzer
that stands out in every technical interview.

BullMQ + Redis

PostgreSQL

ESLint

OSV.dev

Anthropic AI

Railway Deploy

Duration

10 Weeks (2.5 months)

Stack

Node.js / TypeScript · React · PostgreSQL · Redis

Deployment

Railway (two services: API + Worker)

Interview Tier

Strong backend system design signal

01 — Final Baseline Project

INCLUDED	EXCLUDED
✓ Async job queue (BullMQ + Redis) ✓ Worker as separate process ✓ Job lifecycle: PENDING → RUNNING → COMPLETE/FAILED ✓ Retry + failure reason tracking ✓ GitHub OAuth login (JWT session) ✓ Repo submission with idempotency check ✓ ESLint static analysis (JS/TS only) ✓ OSV.dev dependency vulnerability scan ✓ Anthropic AI executive summary ✓ Per-repo quality + security score (0-100) ✓ React dashboard with live polling ✓ Rate limiting (express-rate-limit) ✓ Structured logging (pino) ✓ Temp directory cleanup after every job ✓ Deployed to Railway with live URL	✗ GitHub webhooks (Week 9 bonus only) ✗ Python / Pylint support ✗ Runtime benchmarking ✗ Docker / gVisor sandboxing ✗ RBAC / multi-tenant isolation ✗ Plugin system ✗ SARIF export ✗ Paid APIs beyond Anthropic ✗ Full-text search (Elasticsearch) ✗ SSE push (polling is sufficient)

Why this scope is sufficient for interviews: The async queue architecture is the entire interview centerpiece. A working, deployed pipeline with real analysis output and AI integration is far more impressive than a half-built system with webhooks. Every excluded feature is either a distraction or a scope trap.

02 — Week-by-Week Execution Plan

Wks 1-2
Foundation

Wks 3-5
Async Core

Wks 6-8
Analysis + AI

Wks 9-10
Polish + Ship

WO
1

Foundation — Database & API Skeleton

Routing · PostgreSQL · Migrations · Structured Logging

A CORE GOAL

Initialise monorepo (apps/api, apps/web, packages/shared). Express server with health-check endpoint. PostgreSQL connected. First migration: users, repositories, jobs, reports, findings tables. Wire pino structured logging as middleware from day one. GET /repos/:id returns seeded data from the DB.

B BONUS

Add Zod-based env-var validation: missing vars crash the server at startup with a clear, readable error.

C LEARNING FOCUS

Production project structure (routes → services → repository pattern). Why migrations trump manual CREATE TABLE. What structured logging gives you over console.log.

D REQUIRED TOPICS

- PostgreSQL schema basics — TEXT vs VARCHAR, UUID vs integer PKs
- What an idempotent migration looks like (CREATE TABLE IF NOT EXISTS)
- Express middleware model and async/await in Node.js
- pino logger — request logging, log levels, JSON output

E IMPLEMENTATION DIRECTION

- Use node-pg-migrate or Prisma. Write all five tables now — you won't need to touch migrations again.
- Wire pino as Express middleware: every request logs method, path, status, duration.
- DB connection failure must crash the process with a readable message, not silently fail.
- Put all DB logic in a service module — never inside route handlers.

F EDGE CASES

- Wrong DB connection string → fail fast on startup, not on first request.
- Migration running twice → idempotent CREATE TABLE IF NOT EXISTS.

G PITFALLS

- Creating tables manually in a DB client — always use migration files.
- Planning to 'add logging later' — wire pino on Day 1 or it never happens.
- Putting DB logic inside route handlers — extract to a service layer immediately.

W0 2

Auth & Repo Ingestion

GitHub OAuth · JWT · Idempotency · Rate Limiting

A CORE GOAL

GitHub OAuth login (/auth/github → callback → JWT cookie). User created in DB on first login. POST /repos: accepts GitHub URL, validates format, idempotency check, creates repositories record, creates jobs record with status PENDING, returns { repold, jobId }. GET /jobs/:id returns current DB status. Add express-rate-limit (100 req / 15 min per IP) — this takes 10 minutes and must not be deferred.

B BONUS

Encrypt the GitHub access token before storing (AES-256 with TOKEN_ENCRYPTION_KEY env var).

C LEARNING FOCUS

OAuth 2.0 authorization code flow. JWT structure and expiry. The 'accept now, process later' pattern — the conceptual core of every async system.

D REQUIRED TOPICS

- OAuth 2.0 sequence — draw it on paper: user → server → GitHub → callback → token
- What the state parameter prevents (CSRF)
- JWT: header, payload, signature — why it expires
- How to validate a JWT on subsequent requests

E IMPLEMENTATION DIRECTION

- Implement OAuth manually (40 lines) — teaches more than a library.
- Idempotency: query repositories by owner+name before inserting. If found, return the existing record.
- Rate limiting: app.use(rateLimit({ windowMs: 15*60*1000, max: 100 }))) — one line, done.
- POST /repos handler must validate GitHub URL format before touching the DB.

F EDGE CASES

- Same repo submitted twice → return existing record and its current job status, not a duplicate.
- OAuth state mismatch → return 400 with a clear error, not a silent failure.

G PITFALLS

- Storing GitHub tokens in plain text — know why AES-256 matters even if you skip it.
- Forgetting to validate the state parameter — makes login vulnerable to CSRF.
- Deferring rate limiting to 'later' — it takes 10 minutes and belongs here.

W0 3

Async Job Queue

BullMQ · Redis · Separate Worker Process · Job Lifecycle

A CORE GOAL

Install Redis (Docker locally). Install BullMQ. Create analysis queue. Modify POST /repos to enqueue a BullMQ job. Write worker.ts as a completely separate process that: dequeues jobs, logs via pino, updates DB status PENDING→RUNNING→COMPLETE, writes a stub report (hardcoded scores). GET /jobs/:id now shows live status changes as the worker runs. Manually kill the worker mid-job and verify BullMQ re-queues it.

B BONUS

Configure BullMQ retry with exponential backoff (3 attempts). Jobs that exhaust retries must update DB to FAILED with failure_reason populated.

C LEARNING FOCUS

Why message queues decouple producers from consumers. BullMQ job lifecycle: waiting → active → completed/failed. Why Redis is the broker. Why the worker must be a separate process.

D REQUIRED TOPICS

- BullMQ: Queue, Worker, Job, QueueEvents — read the official docs
- What a 'stalled job' means and how BullMQ handles it
- Redis data types — sorted sets and lists underpin BullMQ (conceptual only)
- What 'at-least-once delivery' means and why it creates idempotency needs

E IMPLEMENTATION DIRECTION

- worker.ts runs as its own Node process — never imported into the API.
- Add failure_reason column to jobs table now — you'll populate it in Week 6.
- Worker uses pino, not console.log.
- Docker Redis: docker run -p 6379:6379 redis
- Test stalled job explicitly: kill -9 the worker mid-job, restart it, confirm the job resumes.

F EDGE CASES

- Redis down when API enqueues → catch, return 503 immediately.
- Worker crashes mid-job → BullMQ stalled detection re-queues it. Test this manually.

G PITFALLS

- Running the worker inside the API process — works but defeats the purpose and the interview point.
- BullMQ job data is JSON-serialised — no class instances, functions, or circular refs.
- Not testing the stalled job scenario explicitly — it's your #1 interview talking point.

W04**React Frontend & Job Status Polling**

Vite · Polling · CORS · End-to-End Demo

A CORE GOAL

Build a minimal React app (Vite). Three screens: (1) Login with GitHub button. (2) Form to submit a repo URL. (3) Job status screen polling GET /jobs/:id every 3 seconds, showing PENDING→RUNNING→COMPLETE with a spinner. When complete, display the stub report. This is your first working end-to-end demo — everything after this is additive.

B BONUS

Replace polling with SSE: GET /jobs/:id/events pushes updates; frontend uses EventSource. Cleaner pattern, worth building if polling is solid.

C LEARNING FOCUS

How frontend and backend communicate during long async operations. The polling pattern and its tradeoffs vs SSE. CORS and why it exists.

D REQUIRED TOPICS

- `useEffect` cleanup with `setInterval` — why the return matters
- What `EventSource` is (browser API for SSE)
- CORS — why `localhost:3000` can't call `localhost:4000` without headers
- credentials: 'include' on fetch for cookie-based auth

E IMPLEMENTATION DIRECTION

- Stop polling when status is `COMPLETE` or `FAILED` — never poll forever.
- Show a meaningful loading state from the moment the form is submitted.
- CORS middleware on Express: `origin: 'http://localhost:3000', credentials: true`.
- UI does not need to be polished this week — it needs to work.

F EDGE CASES

- User closes browser mid-job → job continues, DB reflects truth when they return.
- Expired JWT during polling → redirect to login, not an error page.

G PITFALLS

- Polling forever after job completes — stop the interval on `COMPLETE` or `FAILED`.
- Not handling the initial loading state between form submission and first poll response.

W0

5

Async System Hardening

No New Features — Reliability · Failure Tracking · Worker Heartbeat

A CORE GOAL

This week has zero new features. Goal: make the existing system production-quality before adding real analysis. Specifically: (1) Populate `failure_reason` on DB when jobs fail. (2) Worker heartbeat — worker writes `worker:heartbeat` Redis key every 30 seconds; `/health/worker` endpoint checks this key, returns 503 if stale or missing. (3) Job timeout enforcement — jobs running over 5 minutes are force-failed. (4) Concurrent idempotency test — two simultaneous `POST /repos` for the same repo must produce one job, not two.

B BONUS

Add `GET /queue/stats` endpoint returning BullMQ queue depth, active count, and failed count. Useful for production debugging.

C LEARNING FOCUS

Why hardening before adding features prevents compounding instability. How at-least-once delivery creates idempotency requirements. What a dead-letter queue is conceptually.

D REQUIRED TOPICS

- BullMQ `QueueEvents` for listening to job completions and failures
- Redis `SET` with `EX` (TTL) for heartbeat signals
- What idempotency means in distributed systems
- How BullMQ's `removeOnComplete` and `removeOnFail` options affect job retention

E IMPLEMENTATION DIRECTION

- Worker heartbeat: `setInterval(() => redis.set('worker:heartbeat', Date.now(), 'EX', 60), 30000)`.
- `/health/worker`: reads the key, returns 503 if missing or older than 60 seconds.

- Concurrent idempotency: test with two simultaneous POST /repos calls using Promise.all.
- Failure reason: catch block in worker populates failure_reason before marking FAILED.

F EDGE CASES

- Two simultaneous repo submissions → only one job created. Use a DB unique constraint as the backstop.
- Worker heartbeat key expires (worker down) → monitoring endpoint returns 503, not 200.

G PITFALLS

- Skipping this week because there's 'nothing new to build' — this is the most common failure mode.
- The reliability built here is what separates this project from a toy. Skipping means Weeks 6-8 are unstable.

**W0
6****Static Analysis Engine**

ESLint · child_process.execFile · Subprocess Timeout · Cleanup

A CORE GOAL

In the worker, after dequeuing: (1) Shallow-clone the repo (git clone --depth 1). (2) Run ESLint as a child process via execFile with --format json. (3) Parse JSON output, map findings to the findings table schema. (4) Compute quality score: 100 minus weighted finding count. (5) Write report to DB. (6) Clean up /tmp/stackprobe/{jobId} in a try/finally block — always, on success and failure. GET /jobs/:id now returns a real report with real findings from real repo code.

B BONUS

Add cyclomatic complexity analysis using complexity-report (npm). Write top 5 most complex functions as findings with category: 'complexity'.

C LEARNING FOCUS

How static analysis tools work (AST traversal, rule matching). How to run subprocesses safely in Node.js. Why cleanup is mandatory — not optional — in production.

D REQUIRED TOPICS

- child_process.execFile vs exec — why execFile eliminates shell injection risk
- ESLint --format json output structure: array of file results, each with messages array
- How to enforce a timeout on a child process (AbortController or process.kill)
- try/finally pattern for guaranteed cleanup

E IMPLEMENTATION DIRECTION

- Clone to /tmp/stackprobe/{jobId}. Set 30-second timeout on ESLint subprocess. SIGKILL on timeout.
- Wrap entire analysis in try/finally — finally block deletes the temp directory.
- Cap findings at 500 before writing to DB.
- Populate failure_reason if clone fails (private repo, invalid URL, network error).
- Repo has no JS/TS files → write a report with zero findings and a note, do not fail the job.

F EDGE CASES

- Repo has no JS/TS files → produce minimal report with a note, not a job failure.
- ESLint finds 10,000 findings → cap at 500 before DB write.
- Clone fails (private/invalid) → mark FAILED with specific failure_reason.

G PITFALLS

- Forgetting the cleanup — temp directories fill disk within hours in production.
- Using exec instead of execFile — shell injection risk with user-supplied repo URLs.
- No subprocess timeout — pathological repos can run ESLint indefinitely.

**WO
7****Dependency Vulnerability Scanning**

OSV.dev Batch API · Cache-Aside · CVSS Scoring

A CORE GOAL

After cloning, parse package.json (dependencies + devDependencies). Query the OSV.dev batch API (POST /v1/querybatch) for all packages in one HTTP call. Write vulnerabilities to findings table with category: 'security'. Add a security score to the report (100 minus weighted CVSS severity sum). If OSV API times out (5 seconds), mark security scan as 'incomplete' — do not fail the job. The final report now has two independently scored sections: code quality and security.

B BONUS

Cache OSV.dev responses in Redis with 1-hour TTL (cache-aside pattern). Same package+version is never queried twice within the hour.

C LEARNING FOCUS

How dependency vulnerability scanning works — it's just API lookups against an advisory database. The cache-aside pattern. Why you must never query external APIs in a serial loop.

D REQUIRED TOPICS

- OSV.dev batch API endpoint and request format (POST /v1/querybatch)
- Semantic versioning basics — what ^1.2.0 and ~1.2.0 mean
- CVSS severity levels: critical (9+), high (7-9), medium (4-7), low (<4)
- Cache-aside: check cache → fallback to API → write to cache before returning

E IMPLEMENTATION DIRECTION

- Use the OSV batch endpoint — one HTTP call for up to 100 packages.
- Filter out non-semver strings (git hashes, local paths, * globs) before querying.
- Set 5-second timeout on the OSV API call — degrade gracefully if it fails.
- Batch 500+ dependencies into groups of 100 for the batch endpoint.
- devDependencies → flag vulnerabilities as lower severity than production deps.

F EDGE CASES

- package.json uses * or workspace globs → skip those entries, don't query OSV.
- OSV API down → degrade gracefully, mark security scan incomplete, do not fail the job.
- 500 dependencies → batch into groups of 100 for the batch endpoint.

G PITFALLS

- Querying OSV one package at a time in a loop — 500 sequential HTTP calls, catastrophically slow.
- Not filtering non-semver version strings — OSV will reject them and the whole batch errors.
- Failing the entire job when OSV is unavailable instead of degrading.

**WO
8****AI-Powered Summary**

Anthropic API · Prompt Engineering · Graceful Degradation

A CORE GOAL

After analysis completes: collect the top 10 most severe findings. Send them to the Anthropic API as a structured JSON block with a prompt asking for a 3-paragraph executive summary covering critical issues, why they matter, and 2-3 next steps. Set `max_tokens: 600`. Store the result in `reports.ai_summary`. Display it prominently in the frontend. Wrap in `try/catch` — if the AI call fails, the report is still valid and shows 'AI summary unavailable'. The AI call must never block or fail the rest of the report.

B BONUS

Ask Claude to return a prioritised list of top 3 things to fix first as JSON. Display as a 'Recommended Actions' panel in the dashboard.

C LEARNING FOCUS

How to integrate an LLM into a backend pipeline deliberately. Prompt engineering basics. Why the AI call comes last and why it must degrade gracefully.

D REQUIRED TOPICS

- Anthropic Messages API — `messages.create` endpoint and content response format
- Token counting awareness — keep prompt + findings under 4,000 tokens
- How to handle 429 rate limit errors with exponential backoff
- Why sending findings as a JSON block beats formatted prose for LLM parsing

E IMPLEMENTATION DIRECTION

- Pre-filter to top 10 findings by severity — never send the full list.
- Zero findings (perfect repo) → send a different prompt acknowledging this.
- AI timeout or 429 → catch, set `ai_summary` to null in DB, show 'unavailable' in UI.
- Log the prompt and response in development so you can tune it.
- The AI step is the last step in the worker pipeline — after all other analysis is complete.

F EDGE CASES

- Zero findings → use a different, shorter prompt: congratulate the developer.
- AI API timeout or 429 → catch, degrade, do not fail the job or invalidate the report.

G PITFALLS

- Sending all 500 findings to the AI — blows the context window and costs money.
- Blocking job completion on the AI call — it is best-effort, not a hard dependency.
- Not caching AI responses — identical findings produce identical summaries. Cache on finding fingerprint hash.

**WO
9****Frontend Polish & Production Deployment**

Railway · Two Services · Score Chart · Live URL

A CORE GOAL

Build the full dashboard: repo list, per-repo score history chart (Recharts), findings table filterable by severity, AI summary section, security findings section. Deploy to Railway as two services: api (Express + React static files) and worker. Serve the React dist/ folder via `express.static()` from the API server — one fewer service. Use Railway shared env var groups for `REDIS_URL` and `DATABASE_URL`. Test the full pipeline end-to-end on Railway with a real repo before calling this week done. Share the live URL.

B BONUS

GitHub webhook integration: POST `/webhooks/github` with HMAC-SHA256 signature verification, auto-enqueues analysis on push. Add this only if everything else is working solidly.

C LEARNING FOCUS

How to deploy a multi-service Node.js application. Why the worker is a separate Railway service. How environment variables work in production vs locally.

D REQUIRED TOPICS

- Railway service configuration — deploy from GitHub, shared env vars, service linking
- Serving React dist/ as Express static files (eliminates a third service)
- CORS in production — frontend domain != API domain, update origins
- Worker heartbeat in production — how to verify the worker is alive after deploy

E IMPLEMENTATION DIRECTION

- vite build → `express.static('dist')` in the API server. Two Railway services only.
- Every URL and secret comes from environment variables — no hardcoded localhost anywhere.
- Test with a real public GitHub repo after deploying. Watch the full pipeline run.
- Recharts score history chart: map analysis history to `{ date, score }` data points.

F EDGE CASES

- Hardcoded localhost in codebase → breaks production entirely. Audit before deploying.
- CORS origins must include the Railway-assigned domain, not just localhost.
- Worker not running in production → heartbeat endpoint returns 503, tells you immediately.

G PITFALLS

- Assuming local behaviour transfers to production — it doesn't. Always test end-to-end on Railway.
- Deploying without running a real repo through the full pipeline — do this before calling Week 9 done.
- Adding webhooks before deployment is stable — do webhooks last, or not at all.

**W1
0****README, Tests & Interview Prep**

Mermaid Architecture Diagram · Vitest · Loom Demo · Clean Code

A CORE GOAL

Write a comprehensive README: Mermaid architecture diagram (GitHub renders natively), feature list, tech stack table, local setup instructions, live demo link. Record a 2-minute Loom video demoing the system end-to-end. Write at least 10 unit tests: scoring function, ESLint output parser, OSV response parser, idempotency logic. Fix any bugs found during recording. Clean up dead code and console.log statements.

B BONUS

Add GET /repos/:id/badge returning an SVG score badge. Add GET /repos/:id/report/latest as a public JSON API endpoint. Add Python/Pylint as a second language.

C LEARNING FOCUS

That documentation and testability are engineering skills, not afterthoughts. How to articulate technical decisions under interview pressure — writing the README forces this.

D REQUIRED TOPICS

- Vitest or Jest basics — how to test pure functions
- Mermaid diagram syntax — GitHub renders it natively in README.md
- How to explain async job processing in one sentence (practice out loud)
- What makes a good architecture diagram (boxes, arrows, no clutter)

E IMPLEMENTATION DIRECTION

- Test pure functions first: scoring algorithm, ESLint parser, OSV version range checker.
- Mermaid in README: graph TD with API → Queue → Worker → DB → Frontend flow.
- README is the first thing a recruiter sees — it is part of the project, not meta-work.
- Demo video must show a real repo being submitted and a real report appearing.
- Do not submit a video of a broken demo. Fix the bug and re-record.

F EDGE CASES

- Demo recording exposes a bug → fix it, re-record. This is expected.
- README written in 20 minutes → recruiter notices. Spend real time on it.

G PITFALLS

- Zero tests — interviewers ask 'how did you test this?' Have an answer.
- No README — a repo without a README signals you don't care about communication.
- Skipping the Loom video — a live demo is worth ten bullet points on a resume.

03 — Simplification Strategy

CATEGORY	ITEM	RATIONALE
SKIPPED	GitHub Webhooks (baseline)	Adds event-driven complexity before the core pipeline is proven. Available as Week 9 bonus only.
SKIPPED	Python / Pylint	Multi-language support doubles the subprocess logic surface. JS/TS only in baseline; Python is Week 10 bonus.
SKIPPED	Docker / gVisor sandboxing	Real security isolation but 3+ weeks of DevOps with zero interview payoff.
SKIPPED	RBAC / multi-tenant isolation	Over-engineered for a portfolio project. Single namespace is acceptable at this scale.
SKIPPED	Runtime benchmarking	Requires sandbox infrastructure consuming entire weeks alone.
SKIPPED	Plugin system / SARIF export	Scope traps with no interview signal.
DELAYED	Frontend polish	Minimal UI in Weeks 1-8. Full dashboard in Week 9 only.
DELAYED	Score history chart	Deferred to Week 9 alongside full deployment.
DELAYED	Redis OSV cache	Week 7 bonus — core scan works without it.
SIMPLIFIED	OAuth implementation	Manual (40 lines) — more educational than a library and faster to debug.
SIMPLIFIED	Frontend + API serving	React dist/ served via express.static() — one fewer Railway service.
SIMPLIFIED	SSE push updates	Polling every 3 seconds is sufficient for MVP. SSE is Week 4 bonus only.

04 — Failure Hotspots

Week 3 — BullMQ + Redis Setup

HIGH RISK

- Most common failure: running the worker inside the API process and not understanding why that's wrong.
- Stalled job detection is your #1 interview talking point — test it by force-killing the worker.
- BullMQ's separate process model is conceptually unfamiliar. Plan a full day just to understand it.
- ✓ Read BullMQ docs before writing any code. Draw the producer/consumer split on paper first.

Week 5 — Hardening (Most Likely to Be Skipped)

HIGH RISK

- Most likely week to skip because 'nothing new is being built'. This is the most damaging mistake.
- Without the worker heartbeat and concurrent idempotency test, Weeks 6-8 are built on an unstable foundation.
- Failure reason tracking skipped here means debugging Week 6 failures is significantly harder.
- ✓ Treat Week 5 as mandatory. The reliability built here is what separates this project from a toy.

Week 6 — Static Analysis — Subprocess & Cleanup

MEDIUM RISK

- Forgetting temp directory cleanup: disk fills in hours in production.
- Using exec instead of execFile: shell injection risk with user-supplied URLs.
- No subprocess timeout: ESLint can hang indefinitely on pathological repos.
- ✓ try/finally for cleanup is non-negotiable. Use execFile with args as array. Set 30-second AbortController timeout.

Week 9 — Production Deployment

MEDIUM RISK

- Deployment always takes longer than expected — budget 2 full days.
- Hardcoded localhost values are the #1 cause of silent production failures.
- CORS misconfiguration: Railway domain is different from localhost:3000.
- Worker not running in production and nobody notices until the demo.
- ✓ Audit for localhost before deploying. Test the full pipeline with a real public repo after deploy. Check the heartbeat endpoint.

05 — Execution Pacing

PHASE 1 — FOUNDATION

Weeks 1–2

Establish a working database-backed API with auth and repo ingestion. The goal is to be bored — routing, DB, auth. Boring is correct here. No async complexity yet.

Deliverable: GET /repos/:id returns real data. POST /repos creates a PENDING job. GitHub OAuth works.

- W1: DB + Logging
- W2: OAuth + Rate Limiting

PHASE 2 — ASYNC CORE

Weeks 3–5

This is the heart of the project. The queue, the worker, and the hardening of both. Do not rush through Week 5. The interview-worthy architecture is built here.

Deliverable: BullMQ queue running. Worker as separate process. Job lifecycle visible in UI. Heartbeat monitoring. Concurrent idempotency proven.

- W3: BullMQ + Redis
- W4: React + Polling
- W5: Hardening

PHASE 3 — ANALYSIS + AI

Weeks 6–8

Fill the pipeline with real analysis. Each week introduces exactly one new external system: subprocess execution (W6), OSV API (W7), Anthropic AI (W8). No stacking.

Deliverable: Real ESLint findings in DB. Real CVEs from OSV.dev. Real AI summary in the report. Full analysis pipeline end-to-end.

- W6: ESLint Subprocess
- W7: OSV.dev Scan
- W8: AI Summary

PHASE 4 — SHIP IT

Weeks 9–10

Make the system demonstrable and explainable. Polish the dashboard, deploy to Railway, write the README, record the demo, write 10 tests. The project is done when there's a live URL.

Deliverable: Live Railway deployment. React dashboard with score chart. README with Mermaid diagram. 10+ unit tests. 2-min Loom demo. Public URL to share.

- W9: Deploy + Dashboard
- W10: README + Tests

06 — Final Outcome

What You Will Have Built

A deployed GitHub repository analyzer with a genuine async job pipeline. The API accepts repo URLs and responds immediately with a job ID. A separate worker process runs ESLint as a subprocess, scans dependencies via the OSV.dev batch API, and generates an AI-powered executive summary via Anthropic. Results are stored in PostgreSQL and displayed in a React dashboard with live polling. The system handles worker failures, automatic retries, and idempotent job submission. Two Railway services share a PostgreSQL database and Redis instance. A live URL exists that anyone can visit.

What You Can Say in Interviews

"The API never does slow work. It accepts a repo URL, enqueues a BullMQ job to Redis, and responds in milliseconds with a job ID. A separate worker process picks up the job, runs ESLint as a subprocess, scans dependencies via OSV.dev's batch API, and generates an AI summary via Anthropic — then writes everything to PostgreSQL. If the worker crashes mid-analysis, BullMQ marks the job as stalled and the next worker restart picks it up. The API degrades gracefully if OSV or the AI API is unavailable — the job still completes, just with reduced output. Workers are stateless, so scaling means running more of them with no code changes."

Likely Interview Questions — Prepared Answers

Q: Why a queue instead of processing synchronously?

A: Code analysis takes 30–120 seconds. A synchronous HTTP request would time out. The queue lets the API respond in milliseconds and do the work in the background.

Q: What happens if the worker crashes?

A: BullMQ marks jobs as stalled after a configurable timeout. The next worker restart picks them up. Job status is written to PostgreSQL at each lifecycle step, so there's no ambiguity.

Q: How do you prevent one large repo from blocking everyone?

A: BullMQ supports job priorities. Smaller repos get higher priority so they don't wait behind a monorepo analysis taking 5 minutes.

Q: How would you scale this to 10,000 repos per day?

A: Run more worker processes. Workers are stateless — they pull from Redis and write to Postgres. Add 10 more workers with no code changes. Queue depth is the autoscaling signal.

Q: Why OSV.dev instead of Snyk?

A: OSV.dev is free, open, and covers npm/PyPI/Go. Snyk requires a paid key for commercial use. OSV demonstrates the same architectural pattern without cost.

Q: How did you test this?

A: Unit tests cover the pure functions: scoring algorithm, ESLint output parser, OSV response parser, idempotency logic. I also tested the stalled job scenario manually by force-killing the worker mid-analysis.

Why This Project Stands Out

Async architecture, not another CRUD app

Most student portfolios are to-do lists. This project has the same async pipeline pattern that powers GitHub Actions, Vercel builds, and Stripe payment processing.

Whiteboard-ready design	You can draw the queue → worker → DB flow on a whiteboard, explain every design decision, and answer adversarial follow-up questions. That's rare.
Deliberate AI integration	The LLM sits at the end of the pipeline where it belongs. It degrades gracefully. It solves a real problem. It's not a chatbot bolted on.
Deployed and accessible	A live URL changes the conversation from 'I built this' to 'here, try it'. Most candidates can't say that.
Production patterns demonstrated	Structured logging, rate limiting, idempotency, failure tracking, worker heartbeat — these aren't features, they're signals that you understand production systems.